

LAPORAN PRAKTIKUM STRUKTUR DATA



Oleh:
FAWWAZ KHALID
2511532004

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T
ASISTEN LABOR : M GALID

FAKULTAS TEKNOLOGI INFORMASI

PEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 2026

KATA PENGANTAR

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah Pemrograman Dasar dengan Java. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2025

Tim Penyusun

FAWWAZ KHALID

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Latar Belakang Praktikum.....	1
1.3 Rumusan Masalah.....	1
1.4 Tujuan Praktikum.....	1
BAB II PENULISAN LAPORAN PRAKTIKUM.....	2
2.1 Alat dan Bahan.....	2
2.2 Landasan Teori.....	2
2.3 Langkah Kerja.....	2
2.4 Hasil dan Analisis.....	3
BAB III KESIMPULAN.....	4
3.1 Ringkasan.....	4
DAFTAR PUSTAKA.....	4

BAB I

PENDAHULUAN

1.1 Pengertian ADT

Tipe Data Abstrak (Tda) Atau Lebih Dikenal Dalam [Bahasa Inggris](#) Sebagai **Abstract Data Type (Adt)** Merupakan Model [Matematika](#) Yang Merujuk Pada Sejumlah Bentuk Struktur Data Yang Memiliki Kegunaan Atau Perilaku Yang Serupa; Atau Suatu [Tipe Data](#) Dari Suatu [Bahasa Pemrograman](#) Yang Memiliki Semantik Yang Serupa. Tipe Data Abstrak Umumnya Didefinisikan Tidak Secara Langsung, Melainkan Hanya Melalui Operasi Matematis Tertentu Sehingga Membutuhkan Penggunaan Tipe Data Tersebut Meski Dengan Risiko Kompleksitas Yang Lebih Tinggi Atas Operasi Tersebut.

BAB II

PENULISAN LAPORAN PRAKTIKUM

2.1 Alat dan Bahan

1. Perangkat baik komputer maupun laptop
2. Teks editor atau *IDE* (misalnya NetBeans, Eclipse, atau IntelliJ IDEA)

2.2 Landasan Teori

1. Tipe data sebuah variabel adalah kumpulan nilai yang dapat dimuat oleh variabel ini. Misalnya sebuah tipe boolean hanya bernilai TRUE atau FALSE, tidak boleh nilai yang lain. TDA adalah suatu model matematika, disertai sekumpulan operasi terhadap model itu

2.3 Langkah Kerja

1. Membuat projek JAVA
2. Membuat file baru di dalam Projek JAVA tersebut
3. Masukkan kode yang ingin dijalankan
contoh kode yang dikerjakan di pekan 3

(github saya sedang bermasalah, dan ini saya screenshoot dari codespace)



```
latihanstack_2511532004.java X
pekan3_2511532004 > J latihanstack_2511532004.java
1  package pekan3_2511532004;
2  import java.util.*;
3
4  public class latihanstack_2511532004 {
5
6      public static void main(String[] args) {
7          Stack<Integer> s = new Stack<Integer>();
8          s.push(42);
9          s.push(-3);
10         s.push(12);
11         System.out.println("nilai stack= " + s);
12         System.out.println("nilai pop = " + s.pop());
13         System.out.println("nilai stack setelah pop= " + s);
14         System.out.println("nilai peek " + s.peek());
15         System.out.println("nilai stack setelah peek= " + s);
16     }
17
18 }
19
```

```
1 package pekan3_2511532004;
2 import java.util.ArrayList;
3
4 class siswa {
5     String nama;
6     int nim;
7
8     siswa(String nama, int nim) {
9         this.nama = nama;
10        this.nim = nim;
11    }
12
13    public String toString() {
14        return "Nim : " + nim + ", nama : " + nama;
15    }
16 }
17
18 public class siswastack_2511532004 {
19     private ArrayList<siswa> stack;
20
21     public siswastack_2511532004() {
22         stack = new ArrayList<>();
23     }
24
25     public void push(siswa mhs) {
26         stack.add(mhs);
27     }
28
29     public siswa pop() {
30         if (!isEmpty()) {
31             return stack.remove(stack.size() - 1);
32         }
33         return null;
34     }
35
36     public siswa peek() {
37         if (!isEmpty()) {
38             return stack.get(stack.size() - 1);
39         }
40         return null;
41     }
42
43     public boolean isEmpty() {
44         return stack.isEmpty();
45     }
46
47     public int size() {
48         return stack.size();
49     }
50 }
```

```
J nilaimaksimum_2511532004.java X
pekan3_2511532004 > J nilaimaksimum_2511532004.java
1 package pekan3_2511532004;
2 import java.util.*;
3 public class nilaimaksimum_2511532004 {
4     public static int max(Stack<Integer> s) {
5
6         Stack<Integer> backup = new Stack<Integer>();
7         int maxvalue_2511532004 = s.pop();
8         backup.push(maxvalue_2511532004);
9         while (!s.isEmpty()) {
10             int next = s.pop();
11             backup.push(next);
12             maxvalue_2511532004 = Math.max(maxvalue_2511532004, next);
13         }
14         while (!backup.isEmpty()) {
15             s.push(backup.pop());
16         }
17         return maxvalue_2511532004;
18     }
19 }
20
21 public static void main(String[] args) {
22     Stack<Integer> s = new Stack<Integer>();
23     s.push(70);
24     s.push(12);
25     s.push(20);
26     System.out.println("isi stack " + s);
27     System.out.println("stack teratas " + s.peek());
28     System.out.println("nilai maksimum " + max(s));
29 }
30
31 }
```

```
pekan3_2511532004 > J siswastack_2511532004.java
18 public class siswastack_2511532004 {
19     }
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36     public siswa peek() {
37         if (!isEmpty()) {
38             return stack.get(stack.size() - 1);
39         }
40         return null;
41     }
42
43     public boolean isEmpty() {
44         return stack.isEmpty();
45     }
46
47     public void tampilkanasiswa() {
48         for (int i = stack.size() - 1; i >= 0; i--) {
49             System.out.println(stack.get(i));
50         }
51     }
52
53     public static void main(String[] args) {
54         siswastack_2511532004 studentStack = new siswastack_2511532004();
55
56         siswa mhs1 = new siswa("ali", 1);
57         siswa mhs2 = new siswa("boby", 2);
58         siswa mhs3 = new siswa("charles", 3);
59
60         studentStack.push(mhs1);
61         studentStack.push(mhs2);
62         studentStack.push(mhs3);
63
64         System.out.println("siswa teratas " + studentStack.peek());
65         System.out.println("mengeluarkan siswa teratas dari stack " + studentStack.pop());
66         System.out.println("daftar siswa setelah di pop");
67         studentStack.tampilkanasiswa();
68     }
69 }
```

J stackarray_2511532004.java X

pekan3_2511532004 > J stackarray_2511532004.java

```
1 package pekan3_2511532004;
2 import java.util.*;
3 public class stackarray_2511532004 {
4     static final int MAX = 1000;
5     int top;
6     int a[] = new int[MAX];
7     boolean isEmpty_2511532004() {
8         return (top<0);
9     }
10    stackarray_2511532004(){
11        top = -1;
12    }
13    boolean push_2511532004(int x) {
14        if (top >= (MAX -1)) {
15            System.out.println("stack overflow");
16            return true;
17        }
18        else {
19            a[++top] = x;
20            System.out.println(x + " dimasukkan kedalam stack");
21            return false;
22        }
23    }
24    int pop_2511532004() {
25        if (top<0) {
26            System.out.println("stackunderflow");
27            return 0;
28        }
29        else {
30            int x = a[top--];
31            return x;
32        }
33    }
34    int peek_2511532004() {
35        if (top<0) {
36            System.out.println("stack underflow");
37            return 0;
38        }
39    }
```

```
32    }
33    }
34    int peek_2511532004() {
35        if (top<0) {
36            System.out.println("stack underflow");
37            return 0;
38        }
39        else {
40            int x = a[top];
41            return x;
42        }
43    }
44    void print_2511532004() {
45        for (int i = top;i>-1;i--) {
46            System.out.println(" " + a[i]);
47        }
48    }
49
50
51 }
```

```

pekan3_2511532004 > J stackarraydriver_2511532004.java
1  package pekan3_2511532004;
2
3  public class stackarraydriver_2511532004 {
4      public static void main(String[] args)
5      {
6
7          stackarray_2511532004 s = new stackarray_2511532004();
8          s.push_2511532004(10);
9          s.push_2511532004(20);
10         s.push_2511532004(30);
11         System.out.println(s.pop_2511532004() + " dikeluarkan dari stack");
12         System.out.println("elemen teratas adalah " + s.peek_2511532004());
13         System.out.println("elementy pada stack : ");
14         s.print_2511532004();
15     }
16 }
17

pekan3_2511532004 > J stackpostfixstack_2511532004.java
1  package pekan3_2511532004;
2  import java.util.Scanner;
3  import java.util.*;
4
5  public class stackpostfixstack_2511532004 {
6
7      public static int postfixEvaluate_2511532004(String expression){
8
9          Stack<Integer> s = new Stack<Integer>();
10         Scanner input = new Scanner(expression);
11         while (input.hasNext()) {
12             if (input.hasNextInt()) {
13                 s.push(input.nextInt());
14             }
15             else {
16                 String operator = input.next();
17                 int operand2 = s.pop();
18                 int operand1 = s.pop();
19                 if (operator.equals("+")) {
20                     s.push(operand1 + operand2);
21                 } else if (operator.equals("-")) {
22                     s.push(operand1 - operand2);
23                 } else if (operator.equals("*")) {
24                     s.push(operand1 * operand2);
25                 } else {
26                     s.push(operand1 / operand2);
27                 }
28             }
29         }
30         input.close();
31         return s.pop();
32     }
33
34     public static void main(String[] args) {
35         System.out.println("hasil postfix = " + postfixEvaluate_2511532004("5 2 4 * + 7 -"));
36     }
37 }

```

2.4 Hasil dan Analisis

Jika kode berjalan seperti yang kita inginkan maka:

1. Menyimpan file
2. Mencatat hasil output

Jika kode gagal maka:

1. Mencari kode yang salah lalu coba jalankan ulang

BAB III

KESIMPULAN

3.1 Ringkasan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Deklarasi variabel pada Java harus disesuaikan dengan tipe data
2. Bisa menggunakan queue , stack, linkedlist dan arraylist untuk fungsi berbeda

DAFTAR PUSTAKA

1. Tipe data abstrak-wikipedia
https://id.wikipedia.org/wiki/Tipe_data_abstrak